

算法设计与分析课程考试答案 (B)

.....

一、基础概念简答题（每题 5 分，共 4 小题 20 分）

1. 答： $T(n) = O(f(n))$ ：若存在 $c > 0$ ，和正整数 $n_0 \geq 1$ ，使得当 $n \geq n_0$ 时，总有 $T(n) \leq c * f(n)$ 。（给出了算法时间复杂度的上界，不可能比 $c * f(n)$ 更大）
2. 答：回溯法的基本思想是在一棵含有问题全部可能解的状态空间树上进行深度优先搜索，解为叶子结点。搜索过程中，每到达一个结点时，则判断该结点为根的子树是否含有问题的解，如果可以确定该子树中不含有问题的解，则放弃对该子树的搜索，退回到上层父结点，继续下一步深度优先搜索过程。在回溯法中，并不是先构造出整棵状态空间树，再进行搜索，而是在搜索过程，逐步构造出状态空间树，即边搜索，边构造。
3. 答：当一个问题的最优解包含其子问题的最优解时，称此问题具有最优子结构性质。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。
4. 答：分治法通常采用递归算法涉及技术，在每一层递归上都有 3 个步骤：
 - （1）分解。将原问题分解为若干个规模较小，相互独立，与原问题形式相同的子问题。
 - （2）求解子问题。若子问题规模较小而容易被解决则直接求解，否则递归地求解各个子问题。

(3) 合并。将各个子问题的解合并为原问题的解。

二、分析题（每题 10 分，共 3 小题 30 分）

1. 解： $f(n)=3f(n-1)=3^2 f(n-2)=\cdots=3^n f(n-n)=3^n *5=5*3^n$

2. 答：

解：该算法的基本语句是 $s++$ ，所以有：

$$\begin{aligned} f(n) &= \sum_{i=0}^n \sum_{j=0}^i \sum_{k=0}^{j-1} 1 = \sum_{i=0}^n \sum_{j=0}^i (j-1-0+1) = \sum_{i=0}^n \sum_{j=0}^i j \\ &= \sum_{i=0}^n \frac{i(i+1)}{2} = \frac{1}{2} \left(\sum_{i=0}^n i^2 + \sum_{i=0}^n i \right) = \frac{2n^3 + 6n^2 + 4n}{12} = O(n^3) \end{aligned}$$

则该算法的时间复杂度为 $O(n^3)$ 。

3. 答：当单链表中首结点值为 x 时，该算法返回 0，此时应该返回逻辑序号 1。另外当单链表不存在值为 x 的结点时，该算法执行出错，因为 p 为 NULL 时仍执行 $p=p \rightarrow \text{next}$ 。所以该算法不满足正确性和健壮性。

三、解答题（每小题 8 分，共 4 小题 32 分）

1. 答：

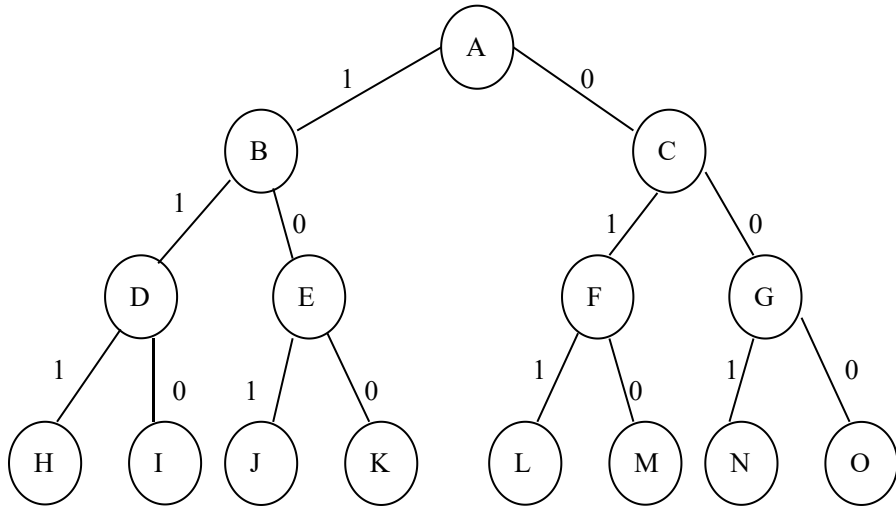
最优方案：(1, 3, 2, 4) 或者 (1, 3, 4, 2)

最优值为：38

2. 答：

解空间为 $\{(0, 0, 0), (0, 1, 0), (0, 0, 1), (1, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 0), (1, 1, 1)\}$ 。

解空间树为：



该问题的最优值为：16 最优解为：(1, 1, 0)

3. 答：将这 10 位客户的申请按照结束时间 $f(i)$ 递增顺序排序，如下表：

i	1	2	3	4	5	6	7	8	9	10
s(i)	0	3	1	5	3	5	11	8	8	6
f(i)	6	5	4	9	8	7	13	12	11	10

i	1	2	3	4	5	6	7	8	9	10
s(i)	1	3	0	5	3	5	6	8	8	11
f(i)	4	5	6	7	8	9	10	11	12	13

1) 选择申请 1 (1, 4)

2) 依次检查后续客户申请，只要与已选择申请相容不冲突，则选择该申请，直到所有申请检查完毕。申请 4 (6, 7)、申请 8 (8, 11)、申请 10 (11,

13)

3) 最后, 可以满足: 申请 1 (1, 4)、申请 4(5, 7)、申请 8 (8, 11)、申请 10 (11, 13) 共 4 个客户申请。这已经是可以满足的最大客户数。

4. 答:

1 2 3 4	5 6 7 8
2 1 4 3	6 5 8 7
3 4 1 2	7 8 5 6
4 3 2 1	8 7 6 5
5 6 7 8	1 2 3 4
6 5 8 7	2 1 4 3
7 8 5 6	3 4 1 2
8 7 6 5	4 3 2 1

四、算法设计题 (共 1 小题, 18 分)

1. 答:

```
void binarysearchtree(int a[], int b[], int n, int **m, int **s, int **w)
{
    int i, j, k, t, l;
    for(i=1; i<=n+1; i++)
    { w[i][i-1]=a[i-1];
      m[i][i-1]=0;}
```

```

for(l=0;l<=n-1;l++)//----l 是下标 j-i 的差

for(i=1;i<=n-1;i++)

{   j=i+1;

    w[i][j]=w[i][j-1]+a[j]+b[j];

    m[i][j]=m[i][i-1]+m[i+1][j]+w[i][j];

    s[i][j]=i;

    for(k=i+1;k<=j;k++)

        {   t=m[i][k-1]+m[k+1][j]+w[i][j];

            if(t<m[i][j])

                { m[i][j]=t;

                    s[i][j]=k;

                }

        }

}

}

}

```